

90 Days SDET Interview Prep

Day 13

How do you start with automation? API vs UI how will you decide?

Manual testing doesn't scale.

When you deploy 10-15 times a day, you can't have humans clicking through everything.

Automation exists to give you speed with confidence.

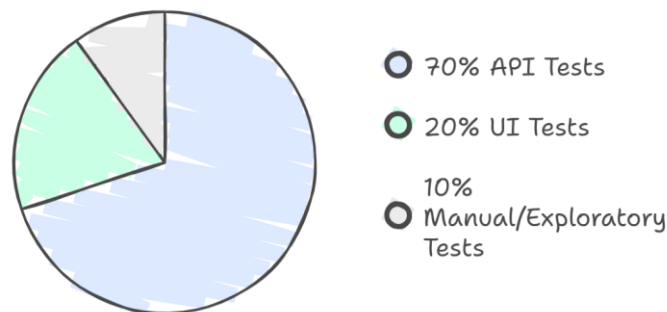
But Exploratory Testing still needs to be done by Human!

Remember:

- The API vs UI decision matters because UI tests are slow, brittle, and expensive to maintain.
- A button not being clickable can break 50 tests.
- API tests are fast, stable, and test actual business logic.

The Correct Approach

Distribution of Test Types in Automation



Always ask where does the real logic live?

Use the Test Pyramid as your framework:

- ~70% API — business logic, validations, data flow
- ~20% UI — user journeys, end-to-end browser flows
- ~10% Manual/Exploratory — edge cases, new features

Start automation on **stable, high-value flows first.**

Never automate a feature whose UI is still actively changing you'll just be rewriting tests every sprint.

2. If the same functionality is testable at both API and UI level, what's your call and why?

- *My call test it at the API level, **with a minimal UI smoke test on top.***
- The reason is simple: API tests are *faster*, more stable, and closer to the business logic. *If I can validate that the data is being created, updated, or deleted correctly at the API layer, I've already covered 80% of the risk.*

The UI test on top just confirms the screen renders it correctly for the user.

- Think of it this way if *your API is returning the right data but the UI is not displaying it, that's a frontend rendering bug which is quick to catch with just one or two UI checks.*
- But if we put all our effort into UI tests, I'm testing through more layers, making my suite slow and brittle.

Test the behavior at API, verify the experience at UI

3 You said API testing is **faster** but if the UI is what the customer uses, isn't UI testing more valuable?

- The customer uses the UI, yes but the value the customer gets comes from the business logic sitting behind it.
- If the logic is broken, no amount of beautiful UI matters. So, API testing protects what truly matters correctness of data and business rules.
- That said, UI testing absolutely has its place. Customer journeys, critical flows like login, checkout, form submission these must have UI coverage because they validate the end-to-end experience a real user would have.
- Both are valuable but serve different purposes. API tests protect business logic efficiently. UI tests protect user experience. A mature strategy needs both just in the right proportion, following the testing pyramid.

4 What if the API is not documented or access is restricted how do you handle API testing then?

- **If not documented:**
 - ✓ You reverse-engineer it.
 - ✓ Open the browser DevTools, -> Network tab-> perform the action on the UI, and capture the API calls being made endpoints, request headers, request body, response structure.
 - ✓ Tools like Postman can import these directly.
 - You build your own documentation as you go.
- **If access is restricted:**
 - ✓ First, you raise it as a risk and escalate to the team lead or architect because testing without API access is a significant gap.
 - ✓ While that's being resolved, you focus on UI-level testing and log the limitation.
 - ✓ If there's a staging or mock environment available, you use that.
 - ✓ Use Tools like WireMock or MockServer to create mock APIs to unblock your testing.

Charles Proxy or Fiddler to intercept API traffic when direct access is restricted

MORE INTERESTING SCENARIO DRIVEN QUESTIONS :

- **UI tests catch issues that API tests can't can you give a concrete example?**
- **How do you decide when contract testing (like Pact) becomes more relevant than traditional API testing?**
- **How do you handle flaky tests early on do you fix them immediately or move forward?**
- **If you join a project mid-way and there's already a messy automation suite, what's your first step?**